Région Hauts-de-France
Cofinancé par l'Union européenne
realconseil DIAGNOSTIC ACCOMPAGNEMENT FORMATION

Notice pratique : Consommer une API REST (fetch et cURL)

1. Appeler une API REST en JavaScript avec `fetch()`

L'appel à une API se déroule toujours en trois étapes :

1. Définir l'URL de la ressource
2. Choisir la méthode HTTP
3. (optionnel) Envoyer des données ou un token d'authentification

---

1.1. Requête GET simple (sans token)

```
fetch("https://api.example.com/pays")
  .then((response) => {
    if (!response.ok) {
      throw new Error("Erreur HTTP : " + response.status);
    }
    return response.json();
  })
  .then((data) => {
    console.log("Liste des pays :", data);
  })
  .catch((error) => {
    console.error(error);
  });
```

Utilisation : Récupérer une liste, lire une ressource, afficher des données.

1.2. Requête GET avec un token (API sécurisée)

Certaines API exigent un **token** dans l'en-tête HTTP `Authorization`.

```
const token = "VOTRE_TOKEN_ICI";

fetch("https://api.example.com/pays", {
  headers: {
    "Authorization": "Bearer " + token
  }
})
```

```
  .then((res) => res.json())
  .then((data) => console.log(data));
```

Deux formats classiques :

- `Authorization: Bearer <token>` (API modernes)
- `Authorization: Token <token>` (APIs plus anciennes)

1.3. Requête POST (création de ressource)

Vous devez envoyer un JSON et déclarer son type dans les headers.

```
fetch("https://api.example.com/pays", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "Authorization": "Bearer VOTRE_TOKEN"
  },
  body: JSON.stringify({
    name: "Gondor",
    capital: "Minas Tirith",
    population: 1500000
  })
})
  .then((res) => res.json())
  .then((data) => console.log("Nouveau pays créé :", data))
  .catch((err) => console.error(err));
```

1.4. Requête PUT (remplacement complet)

```
fetch("https://api.example.com/pays/gondor", {
  method: "PUT",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    name: "Gondor",
    capital: "Osgiliath",
    population: 1200000
  })
});
```

1.5. Requête DELETE

```
fetch("https://api.example.com/pays/gondor", {
  method: "DELETE",
  headers: {
    "Authorization": "Bearer VOTRE_TOKEN"
  }
})
  .then(res => console.log("Suppression :", res.status));
```

---

2. Appeler une API REST avec cURL (terminal)

cURL est idéal pour tester rapidement une API côté backend.

2.1. GET simple

```
curl -X GET https://api.example.com/pays
```

2.2. GET avec token d'authentification

```
curl -X GET https://api.example.com/pays \
  -H "Authorization: Bearer VOTRE_TOKEN"
```

2.3. POST avec JSON

```
curl -X POST https://api.example.com/pays \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer VOTRE_TOKEN" \
  -d '{
        "name": "Rohan",
        "capital": "Edoras",
        "population": 500000
      }'
```

2.4. PUT

```
curl -X PUT https://api.example.com/pays/rohan \
  -H "Content-Type: application/json" \
  -d '{
        "name": "Rohan",
```

```
          "capital": "Aldburg",
          "population": 450000
        }'
```

2.5. DELETE

```
curl -X DELETE https://api.example.com/pays/rohan \
  -H "Authorization: Bearer VOTRE_TOKEN"
```

---

3. Synthèse pratique : fetch vs cURL

Objectif	fetch (JS)	cURL (terminal)
Lire une ressource	`fetch(url)`	`curl url`
Authentification token	headers.Authorization	-H "Authorization: Bearer TOKEN"
Envoyer du JSON	body + headers	-d + -H "Content-Type"
Tester une API rapidement	Moyen	Excellent
Utiliser dans navigateur / frontend	Oui	Non

---

4. Exemple complet (scénario Seigneur des Anneaux)

Objectif : récupérer la liste des personnages

Avec fetch

```
const token = "1234-ABCD-5678";

fetch("https://api.arda.com/personnages", {
  headers: {
    "Authorization": "Bearer " + token
  }
})
  .then(res => res.json())
  .then(data => console.log("Personnages :", data));
```

Avec cURL

```
curl https://api.arda.com/personnages \
  -H "Authorization: Bearer 1234-ABCD-5678"
```